CASE STUDY 42

Student Marks Matrix Analysis

1. PROBLEM STATEMENT

Analyse a matrix of student marks across subjects. Calculate each student's total and average, subject-wise averages,

identify the top performer, and find subjects with low average marks.

2. OBJECTIVES

• Represent student marks using a nested list.

• Calculate row-wise and column-wise statistics.

• Identify the top student and subjects with low averages.

3. ASSUMPTIONS

• Each row represents one student and each column represents one subject.

• Marks are out of 100.

• The subject average is considered low when it is below 50.

• The top performer is the student with the highest total marks.

4. ALGORITHM

1. Create the student names, subject names and marks matrix.

2. For each student row, calculate total and average marks.

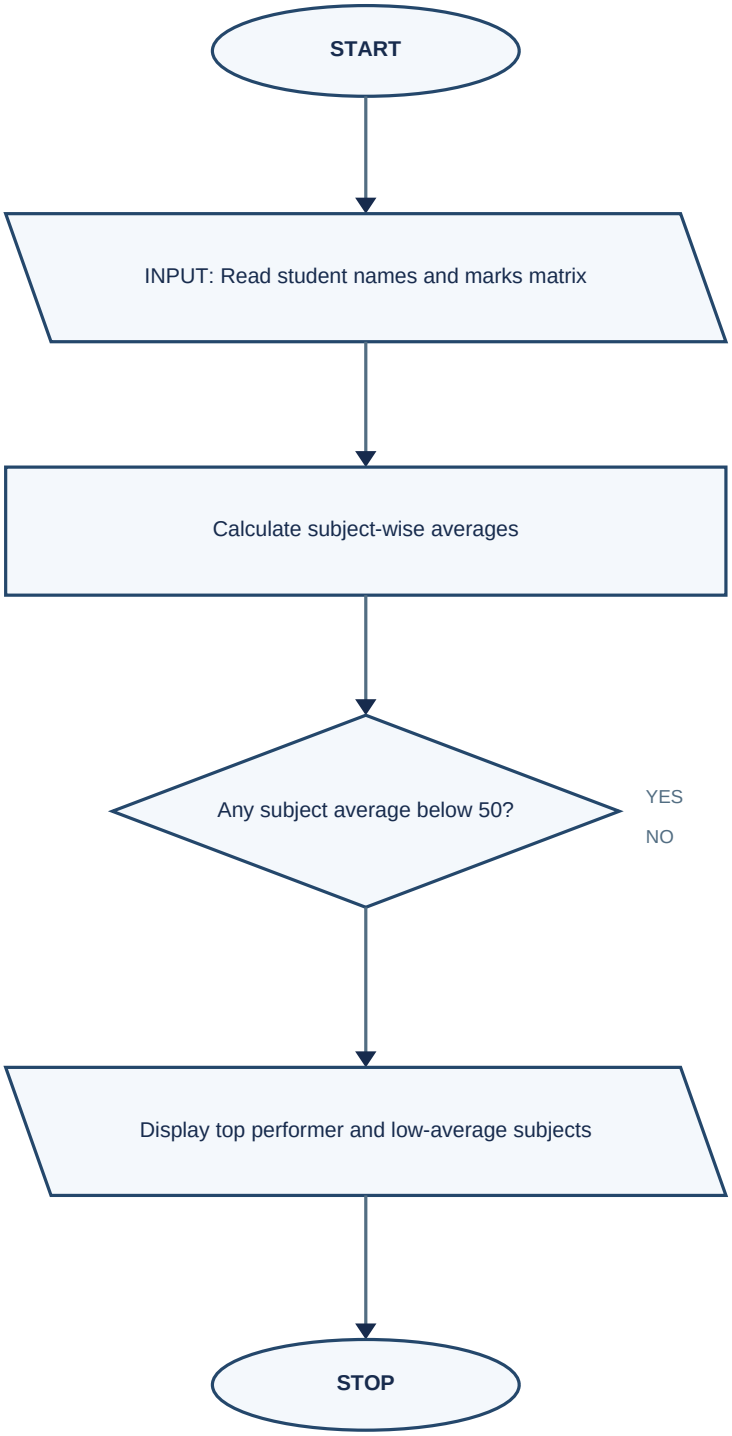
3. For each subject column, calculate the average across all students.

4. Identify any subject whose average is below 50.

5. Find and display the student with the highest total and any low-average subjects.

5. FLOWCHART

Oval: START/STOP | Parallelogram: input/output | Rectangle: process/calculation | Diamond: decision



6. PSEUDOCODE

```
BEGIN
INPUT student names, subjects and marks matrix
FOR each student row
CALCULATE total and average
STORE total and DISPLAY student result
FOR each subject column
CALCULATE subject average
IF average < 50: DISPLAY low-average warning
FIND student with maximum total
DISPLAY top performer and highest total
END
```

7. PYTHON SOURCE CODE

```
students = ["Anu", "Balu", "Chitra"]
subjects = ["Maths", "Physics", "Python"]
# Rows are students; columns are subjects
marks = [
[80, 70, 90],
[60, 45, 75],
[40, 50, 65]
]
totals = []
print("STUDENT-WISE RESULTS")
for i in range(len(marks)):
total = sum(marks[i])
average = total / len(subjects)
totals.append(total)
print(students[i], "Total:", total,
"Average:", round(average, 2))
print("\\nSUBJECT-WISE AVERAGES")
for j in range(len(subjects)):
column_total = 0
for i in range(len(marks)):
column_total = column_total + marks[i][j]
subject_average = column_total / len(marks)
print(subjects[j], "Average:", round(subject_average, 2))
if subject_average < 50:
print("Low average subject")
top_index = totals.index(max(totals))
print("Top performer:", students[top_index])
print("Highest total:", totals[top_index])
```

8. TEST CASES & EXPECTED RESULTS

TEST INPUT / CONDITION REASONING EXPECTED RESULT

Anu: 80, 70, 90 Total 240; average 80 Correct row statistics

Maths: 80, 60, 40 Average 60 Not a low-average subject

Python: 90, 75, 65 Average 76.67 Not a low-average subject

9. SAMPLE OUTPUT

```
STUDENT-WISE RESULTS
Anu Total: 240 Average: 80.0
Balu Total: 180 Average: 60.0
Chitra Total: 155 Average: 51.67
SUBJECT-WISE AVERAGES
Maths Average: 60.0
Physics Average: 55.0
Python Average: 76.67
Top performer: Anu
Highest total: 240
```

10. CONCLUSION

The nested-list program analyses student marks row by row and subject by subject, and identifies the

highest-scoring

student.

SYLLABUS CONCEPTS Variables • Data Types • Operators • Conditions • Loops • Lists • Nested Lists • Tuples • String Operations